# DIGITAL LOGIC DESIGN

## The Foosball Legends

**Presented By**

**Muhammad Hamza Abbas**
**Muhammad Ali Anwar**
**Muhammd Hashim Memon**
**Laiba Munib**


**Dhanani School of Science and Engineering**

**Habib University**

**Spring Semester**

**Contents:**

## SECTION-1: INTRODUCTION

Our mission is to develop a captivating foosball game that maximizes the capabilities of the Basys 3 board. This engaging two-player experience puts players in control of two bars each, maneuvering them with precision using keyboard inputs. With interactive features like a quick-reset button for seamless restarts and a screen toggle option for added convenience, players can stay focused on the excitement of the game. Each round begins with anticipation as players press the enter key on the start screen, and the competitive energy ramps up with the spacebar hit that kicks off gameplay.

Behind the scenes, our project architecture is meticulously designed, with several crucial modules seamlessly integrated. The main module serves as the mastermind behind game control, orchestrating smooth transitions and interactions. The VGA module brings the game to life with vibrant on-screen displays, while the animation generator module adds dynamic movement to the bars and ball. Player inputs are efficiently captured by the keyboard module, and the seven-segment module ensures clear and concise display of game information on the Basys 3 board's seven-segment display. With these components working together harmoniously, our foosball game promises an immersive and thrilling gaming experience for players of all levels.

## SECTION-2: USER FLOW DIAGRAM:

→Here is our proposed idea, as outlined in our project proposal.

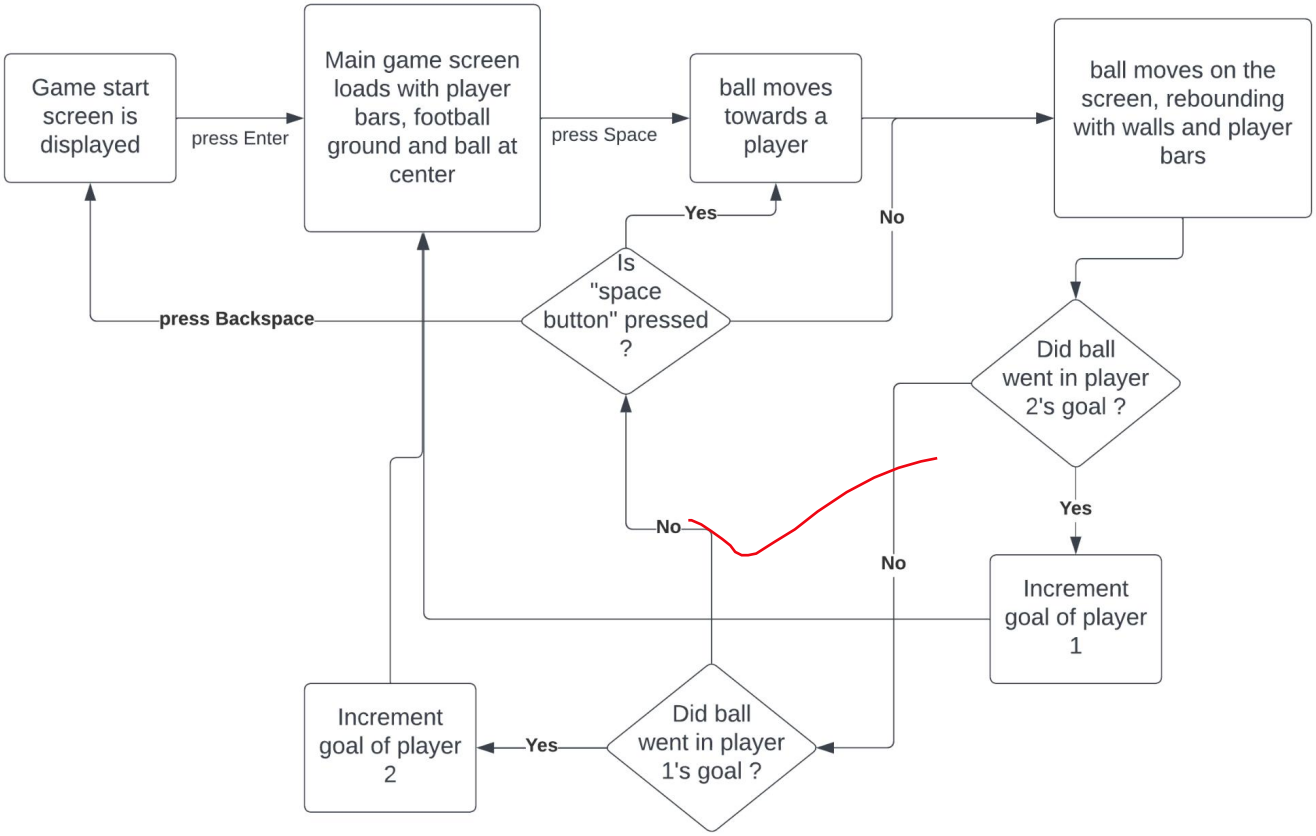



Figure 1: Proposed idea

## SECTION-3: INPUT BLOCK

The user will interact with the game using a keyboard. The keystroke of each playable key of the keyboard will be translated on the display screen as the movements for the player bars, but it will only be vertical i.e. in top and bottom directions.

### 3.1. Keyboard

Foosball Legends employs PS/2-style keyboards, communicating key press data through scan codes. Each key press is associated with a specific code, transmitted when the key is pressed. In the case of a key being held down, the scan code is sent repeatedly at intervals of approximately 100ms. Upon key release, an F0 key-up code is transmitted, followed by the scan code of the released key.

Data transmission from the keyboard to the host occurs in 11-bit words, consisting of a '0' start bit, followed by 8 bits of the scan code (LSB first), an odd parity bit, and concluding with a '1' stop bit. The keyboard generates 11 clock transitions at a frequency of 20 to 30 KHz during data transmission, with the data being valid on the falling edge of the clock. [1]

The scan codes of keyboard and the keys used are given below:

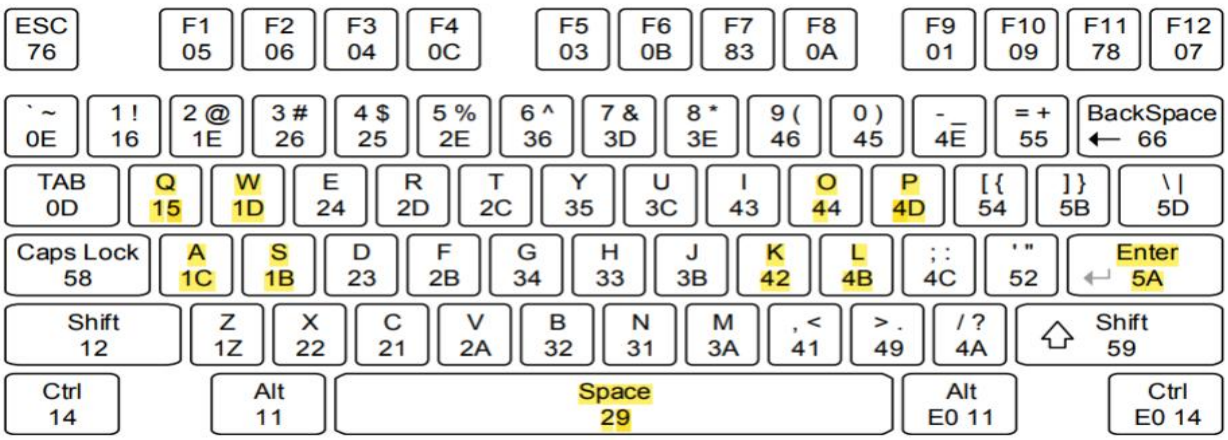



*Figure 9. Keyboard scan codes.*

**3.2: Code Snippets:**

```
module keyboard_1(
    input wire clk, reset,
    input wire ps2d, ps2c,
    output wire start_ball, KEY_UP_4, KEY_DOWN_4, KEY_UP_3, KEY_DOWN_3, KEY_UP_2,
KEY_DOWN_2, KEY_UP_1, KEY_DOWN_1,
    output reg key_release,
    output  reg state
    );

// Declare variables
wire [7:0] dout;
wire rx_done_tick;
supply1 rx_en; // always HIGH
reg [7:0] key;


// Instantiate models
// Nexys4 converts USB to PS2, we grab that data with this module
ps2_rx ps2(clk,
, ps2d, ps2c,
            rx_en, rx_done_tick, dout);

// Sequential logic
always @(posedge clk)
begin
    if (rx_done_tick)
    begin
        key <= dout; // key is one rx cycle behind dout
        if (key == 8'hf0) // check if the key has been released
            key_release <= 1'b1;
        else
            key_release <= 1'b0;
    end
end
```

```
// Output control keys of interest
//assign reset = ((key == 8'h2D) & !key_release) ? ~reset : reset; // 1C is the code
for 'R'
assign start_ball = (key == 8'h29) & !key_release; // 1C is the code for 'Space Bar'
```

```
assign KEY_UP_4 = (key == 8'h4D) & !key_release; // 1C is the code for 'P'
assign KEY_DOWN_4 = (key == 8'h4B) & !key_release; // 1F is the code for 'L'
assign KEY_UP_3 = (key == 8'h44) & !key_release; // 20 is the code for 'O'
assign KEY_DOWN_3 = (key == 8'h42) & !key_release; // 32 is the code for 'K'
assign KEY_UP_2 = (key == 8'h1D) & !key_release; // 1C is the code for 'W'
assign KEY_DOWN_2 = (key == 8'h1B) & !key_release; // 1F is the code for 'S'
assign KEY_UP_1 = (key == 8'h15) & !key_release; // 20 is the code for 'Q'
assign KEY_DOWN_1 = (key == 8'h1C) & !key_release; // 32 is the code for 'A'

always @(posedge clk ) begin
     begin
         // Check if the key is pressed and assign the state accordingly
         case(key)
             8'h5A : state <= 1'b1; // Set state to 1 when Enter key is pressed
             8'h66 : state <= 1'b0; // Set state to 0 when Backspace key is pressed
             default: state <= state; // Maintain state otherwise
         endcase
    end
end


endmodule
```

**SECTION 4-CONTROL BLOCK:**

**4.1-DESCRIPTiON:**

The game consists of two main states which are represented by two different screen

displays:

1. Start Screen

2. Game Screen

**4.2: Start Screen:**

The initial display upon game launch signals the commencement of gameplay. The module responsible for governing this start screen functionality is PixelGen, albeit in its current state, it merely presents a blank screen at the onset of the game as it is yet to be developed. As per the state transition protocol, the start screen materializes automatically upon game initialization. To progress from this start screen to the main game interface, the player is required to input the "Enter" key, thereby effecting the transition. Once within the main game screen, the user retains the option to revert to the start screen by utilizing the "Backspace" key.

**4.3: Game Screen:**

Fig 8: Game Screen

This is the screen where the whole gameplay is based. It is dependent on the **Main controller** Module consisting of **4** main modules: **sync_mod, anim_gen, and SevenSegment**.

The *Top* module takes in a 100 MHz onboard clock and a reset signal. It outputs VGA horizontal sync and vertical sync signals, 4-bit VGA Red, Green, and Blue color values, and receives inputs for the keyboard with some debug led outputs. The module instantiates three sub-modules: **sync_mod** for display timing and resolution, **anime_gen** for main game display, and **SevenSegment** to display the score of each player. The top module assigns the color values based on output received by anim_gen.

i. Sync_mod Module:

The *sync_mod* module defines the timings for a 640x480 pixel resolution display at 60Hz 8with a pixel clock of 25MHz, and provides signals for horizontal sync, vertical sync, active pixel drawing, current pixel position, and pixel clock. **horizontal_scan** is the h_counter, **vertical_scan** is the v_counter while **x_control** and **y_control** are the pixel x and pixel y positions respectively.

ii. anim_gen Module:

The *anim_gen* module is responsible for displaying the main game. The code defines the starting position for ball, goals, bars, size, and movement based on input from the keyboard. It handles all the logic for collision and score updates however it is still incomplete as a part of remains that is responsible for the movement of ball and the corresponding actions.

iii. SevenSegment Module

The *sevenSegment* module is responsible for displaying the score on fpga's build in seven segment display. It just utilizes the value of score of each player which gets updated in anim_gen and displays that number on fpga by first converting it to a bcd format and then utilizing the seven segment display.

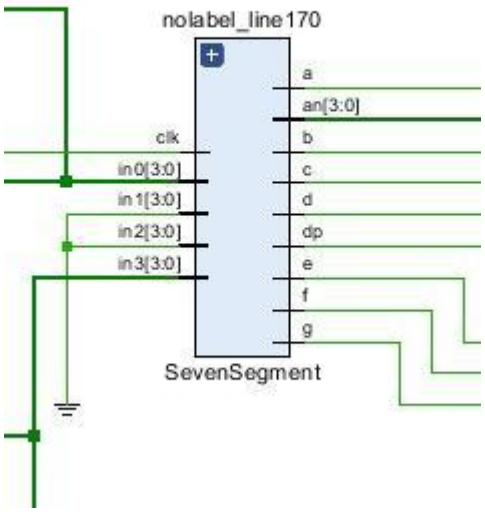



Figure 9: SevenSegment module

**State Transition:** When the player pushes the "Backspace" key, they will be redirected to the start screen. Thus the game returns to its original state.

## SECTION 5- OUTPUT BLOCK:

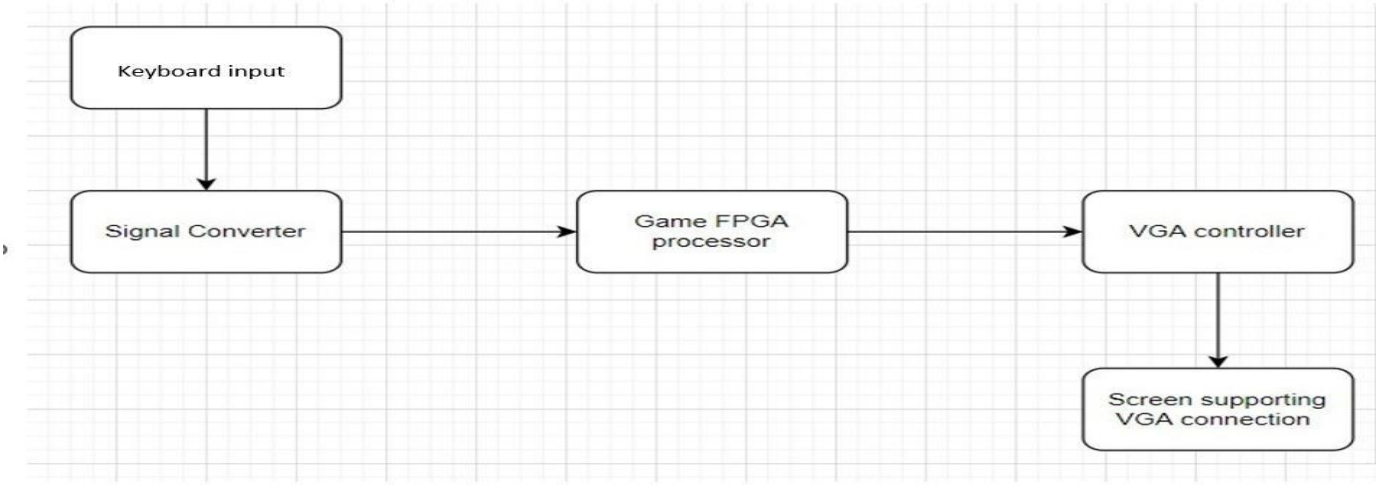
### 5.1-Display Screen:

The display screen is generated using the VGA (video graphic array) connector. The display which we are using is standard 640 x 480 pixels. For imaging on the horizontal axis, we turn the video on from 0 till 639 pixels and for the vertical axis the video is on from 0 till 479 pixels as that is the main display, after that the video is turned off for borders and retracing. To make the final screen we first used a clock divider called clock wiz (The clk_wiz_0 block is provided by Xilinx. Instead of using a clock counter to be more precise we used Vivado's own clk API clk_wiz_0 [2]) to reduce the frequency from 100 MHz to 25 MHz for FPGA to work, then we used h counter to count the pixels of horizontal axis and counter for vertical axis. Then VGA sync was used for turning the video on on the required part of the screen and finally pixel gen and anime gen are used to create our desired screen. Following pin configuration is used to connect the VGA connector to the FPGA board.


Figure 21: Top module Block diagram

**SECTION 6: FSM**

In Foosball Legends, the game operates with a dynamic start screen that is presented upon loading. The transition from the start screen to the main game is initiated by pressing the "Enter" key. During gameplay, the user can navigate back to the start screen by pressing the "Backspace" key. Additionally, a reset button, associated with the V16 pin, enables the user to reset the game when turned off and on again. Furthermore, we can conclude that our game exhibits a **Mealy** Machine behavior by considering user input as a factor in determining the next state. This design choice provides a more interactive and responsive gaming experience.




Game design overview

**SECTION 7-REFERENCE:**

write in IEEE format.

[1] Basys3 fpga user manual, *basys3:basys3_rm.pdf (digilent.com)
[2] clock_wiz_0 module: Clocking Wizard (xilinx.com)
*Image taken from the web or some other resource
[3] The history of Pong, the arcade classic that changed the game - The Verge
[4] https://www.gametablesonline.com/images/P/Foos-200_1.jpg